

Група А

Име: _____ ФН: _____ спец.: _____ група: _____

Трябва да симулирате работата на иновативен калкулатор. За разлика от стандартните калкулатори, този ви позволява да напишете цял аритметичен израз и след това да го редактирате с помощта на курсор, преди да преминете към изчисляването му.

Вашата програма трябва да получава един първоначален израз. В него курсорът се намира в края на израза (най-вдясно). След това получавате поредица от команди за редактиране. В произволен момент можете да поискате да изчислите стойността на израза.

Поддържани елементи в израза:

- **цели числа (включително отрицателни);**
- **оператори:** + (събиране), - (изваждане), * (умножение), / (целочислено деление), % (остатък по модул);
- **скоби:** (и);
- **изразът няма да съдържа други символи.**

Команди за редактиране:

- **type <char>:** На позицията на курсора се вмъква дадения символ (char). Курсорът се премества след нововъведения символ. Символът трябва да бъде допустим елемент за израза;
- **delete:** Изтрива символа непосредствено преди курсора, който не се премества. Ако няма такъв символ, командата не прави нищо;
- **left:** премества курсора с една позиция наляво. Курсорът не може да излезе извън границите на израза;
- **right:** премества курсора с една позиция надясно. Той не може да излезе извън израза;
- **reset:** премества курсора в края на низа (след последния символ);
- **eval:** пресмята текущия израз и извежда стойността му на екрана. Съобщава, ако има грешки в низа (синтактични или семантични). Редактирането трябва да продължи и след това;
- **exit:** прекратява програмата.

Всички операции, с изключение на eval и exit трябва да са с константна сложност; eval трябва да е с линейна сложност по време и памет.

Правила за изчисляване:

- операторите и скобите имат стандартния приоритет и всички са ляво асоциативни;
- може да има унарни + и - Например валиден израз е "5*---1", като стойността му е -5;
- ако входът е невалиден или при поискано изчисление изразът е синтактично невалиден (напр. несдвоени скоби, липсващ операнд и т.н.) изведете подходящо съобщение. Програмата трябва да продължи коректно работа.

Вход:

Всички данни се четат от стандартния вход. На първия му ред ще получите низ - началния израз. Той няма да е по-дълъг от 1023 символа. Следват командите - по една на ред.

Забележки и изисквания:

- съхранете израза в **подходящо избран тип данни. Съблюдавайки изискванията.** Не се изисква реализация на клас за този тип – можете например да работите със свободни функции, получаващи подходящи аргументи;
- позволено е използването на `std::stack`, `std::string`, `std::stringstream`. Не е позволено използването на други готови реализации на структури от данни или алгоритми. Предполага се да реализирате подходящи структури или необходимите ви части от тях.
- вашата програма трябва да гарантира поне базова сигурност при възникване на изключения.

Група Б

Име: _____ ФН: _____ спец.: _____ група: _____

Трябва да реализирате ядрото на текстов редактор за математически изрази и модул за тяхното изчисляване. За целта трябва да използвате **подходящ контейнер за израз**. Дизайнът и операциите свързани с него са ваша отговорност. Не се изисква реализация на клас – свободни сте да работите със свободни функции, получаващи подходящи аргументи. Разрешено е използването на структурите от `<stack>`, `<string>`, `<stringstream>` библиотеките, но **не** и **други**.

Програмата работи в две фази:

1. **Редактиране:** Потребителят въвежда команди за промяна на израз. За целта имате позиция на която се работи – курсор.
2. **Изчисляване:** След приключване на редакциите, изразът се оценява.

За цялостната програма трябва да реализирате следните функции:

1. toList

Функцията приема низ, съдържащ математически израз и го трансформира в подходящ свързан списък, като всеки символ или цифра се съдържа в една клетка на списъка. Входният низ може да съдържа произволен брой празни символи, които не трябва да се съхраняват в получения списък. Резултатът от тази функция ще се използва при извикване следващите две функции:

2. edit

Функцията получава указател към клетката от списъка, създаден от предната функция, която съдържа първия му символ и въвежда и изпълнява командите от режим **Редактиране**. В този режим има курсор, който в началото се намира преди **първия символ** от израз.

Поддържат се следните команди:

- **add <symbol>** - вмъква символ след (в дясно от) курсора;
- **delete** - изтрива един символ след (в дясно от) курсора;
- **move <l(left)/r(right)>** - придвижва курсора с една позиция в подадената посока;

При опит за преместване извън израз (напред или назад) курсорът трябва да се позиционира циклично в другия край на израз.

- **done** – прекратява режима на редакция и връща получения от нея списък с израз;

Всички операции трябва да са константни откъм време и памет!

3. solve

Функцията получава указател към първата клетка от списъка и изчислява израз, описан в него.

Резултатът трябва да е цяло число. За изчислението са зададени следните ограничения и насоки:

1. В израз трябва да има само цели числа (включително отрицателни), оператори (+, -, * и /) и скоби;
2. * и / са с по-висок приоритет от + и -, като всяка двойка оператори е с еднакъв приоритет и всички са ляво асоциативни;
3. В израз може да има унарни оператори при следните правила:
 - **Минус (-)** Ако преди него има оператор или отваряща скоба, а след него има **число**, това се третира като знака на отрицателно число (напр. $1+5$ се чете като $1 + (-5)$).
 - **Деление (/)**: Ако на дадено място, се очаква операнд преди / (имате унарно делене), автоматично се вмъква делимо 1 преди него (напр. $1+2$ се изчислява като $1 + 1/2$).
 - **Плюс (+) и Умножение (*)**: Третират се аналогично на – и / (унарен + и умножение по 1).

При изчисляването на израз следете за всякакви грешки и при откриването на такива съобщете по подходящ начин на извикващия чрез изключение.

Реализирайте главна функция, която прочита символен низ от стандартния вход – един израз, след това го преобразува в списък, който минава през режим редакция и след нейния край – през изчисление. Обработете подобаващо всички грешки. От вашата програма се изисква минимум базова сигурност при възникване на изключения.